

CLOUD COMPUTING

LAB NO # 15



SUBMITTED TO:

Sir Shohaib

SUBMITTED BY:

Name: Amina Noor

Reg No. 2023-BSE-007

Semester: V-A

CLOUD COMPUTING

LAB NO # 15

Project Introduction

In this lab project, I built a small **multi-tier web architecture** on AWS. I used **Terraform** to create the infrastructure and **Ansible roles** to automatically configure the servers.

The goal was to **separate infrastructure and configuration**, like in a real production environment. Terraform created all AWS resources (VPC, subnet, security groups, EC2 instances), while Ansible installed and set up software on the servers.

The system has:

- **1 frontend server** running **Nginx**
- **3 backend servers** running **Apache HTTPD**

The frontend server acts as a **reverse proxy and load balancer**, forwarding requests to backend servers using a **2-active + 1-backup high-availability setup**.

Everything is fully automated: **one Terraform apply command** provisions and configures all resources without manual work.

Project Objectives

- Design AWS infrastructure using **Terraform** in a **modular and clean way**
- Configure frontend and backend servers using **Ansible roles**
- Implement **Nginx load balancing** with **failover**
- Integrate Terraform and Ansible so **no manual Ansible commands are needed**
- Maintain **proper project structure and documentation** for real deployments

Architecture Overview

- The project uses a **frontend–backend model**.
- All **EC2 instances** are in one **VPC** and **public subnet**.
- Each instance has a **public IP** for testing.
- **SSH** is allowed only from my IP; **HTTP** is open for testing.

Load Balancer Behavior:

- Two backend servers handle traffic normally (**round-robin**).
- One backend server acts as a **backup**.
- If the main backends stop, traffic automatically goes to the backup server.

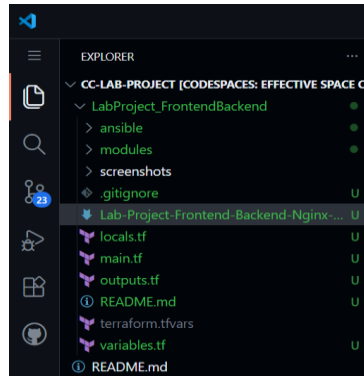
Environment Setup

- I used **GitHub Codespace** as my working environment.
- Codespace provided a **Linux environment** where I installed:
 - **Terraform**
 - **AWS CLI**
 - **Ansible**
 - **Python**

```
Terraform: v1.14.3
AWS CLI: 1.44.24
Ansible: 2.16.3
Python: 3.12.1
```

✅ All tools are installed and verified!

I also **created a GitHub repository** following the required naming rules and kept **clean version control** throughout the project.



Terraform Configuration – Variables and Locals

I defined reusable values in **Terraform variables** instead of hard-coding them. This made the setup flexible.

Variables I created:

- **VPC CIDR block**
- **Subnet CIDR block**
- **Availability zone**
- **Environment prefix**
- **Instance type**
- **SSH key paths**

```
variables.tf U X
LabProject_FrontendBackend > variables.tf
1 # VPC and Subnet Variables
2 variable "vpc_cidr_block" {
3     description = "CIDR block for the VPC"
4     type        = string
5 }
6
7 variable "subnet_cidr_block" {
8     description = "CIDR block for the public subnet"
9     type        = string
10 }
11
12 variable "availability_zone" {
```

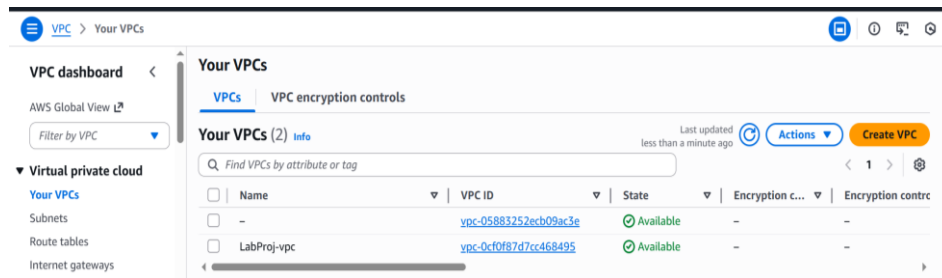
To make it more secure, I used **Terraform locals** to automatically find my public IP and **allow SSH access only from that IP**.

```
variables.tf U locals.tf U X
LabProject_FrontendBackend > locals.tf
1
2 # Get your public IP for SSH security group
3 data "http" "my_ip" {
4     url = "https://icanhazip.com"
5 }
6
7 locals {
8     my_ip = "${chomp(data.http.my_ip.response_body)}/32"
9 }
```

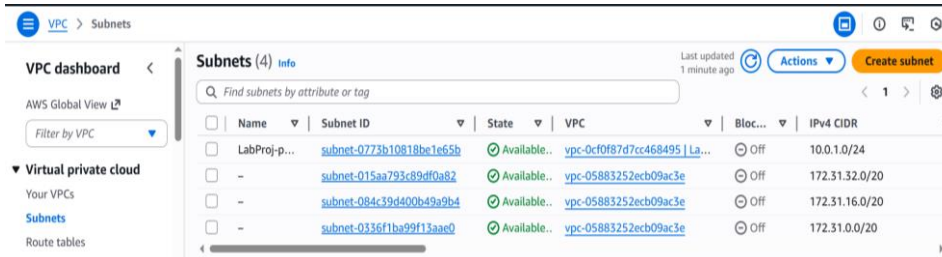
Terraform Networking Setup

I set up the AWS network by creating:

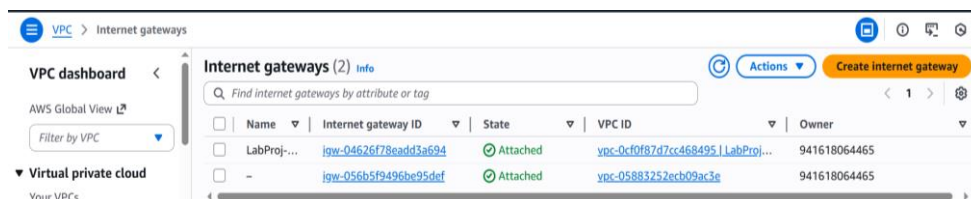
- A **VPC** using the CIDR block from the variables.



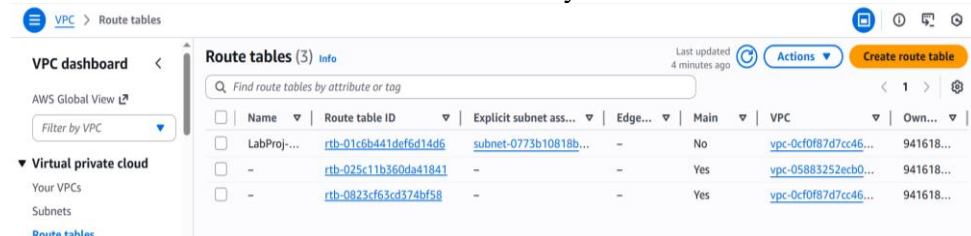
A public subnet inside the VPC



An Internet Gateway attached to the VPC



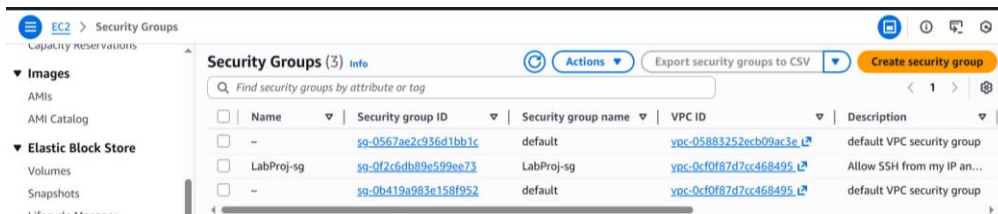
A route table with a default route to the Internet Gateway

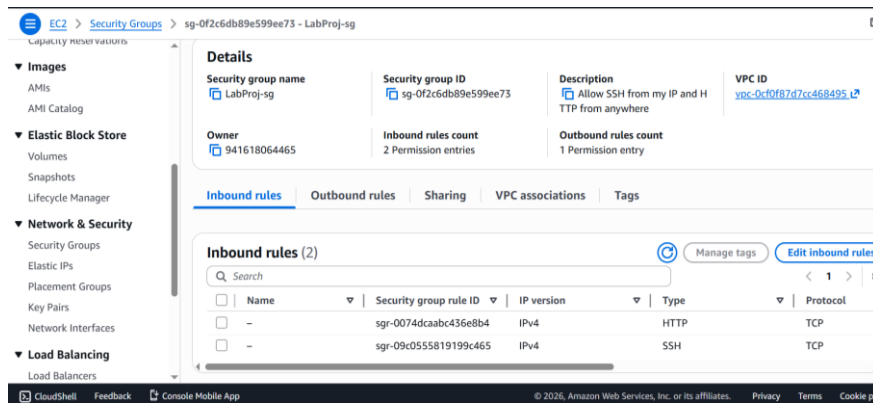


Security Group Configuration

I set up a security group with specific rules:

- **SSH (port 22):** Only my public IP can access.
- **HTTP (port 80):** Open for everyone.





Terraform EC2 Provisioning

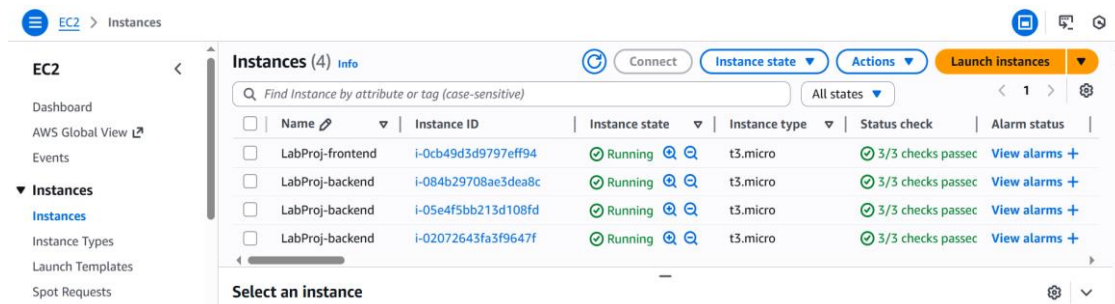
After setting up the network, I created the EC2 instances:

Frontend Instance:

- Created **1 EC2 instance**.
- Clearly **tagged as the frontend server**.
- This server will run **Nginx using Ansible**.

Backend Instances:

- Created **3 EC2 instances** using a **count method**.
- Each instance got a **unique tag**.



Terraform Outputs

I set up **Terraform outputs** to make it easier to check the setup.

These outputs showed:

- **Frontend public IP**
- **Backend public IPs**
- **Backend private IPs**

After defining them, I ran:

terraform apply

This helped me **quickly see all server IPs** without logging into each one.

```
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

backend_private_ips = [
  "10.0.1.155",
  "10.0.1.67",
  "10.0.1.253",
]
backend_public_ips = [
  "3.29.129.170",
  "40.172.186.177",
  "51.112.52.18",
]
frontend_public_ip = "3.28.45.9"
```

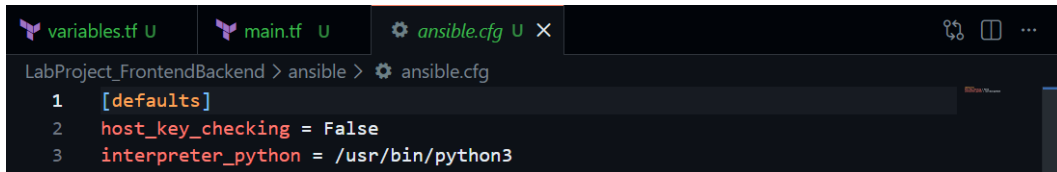
Ansible Global Configuration

Before using roles, I set up **Ansible's global settings**.

I updated the **ansible.cfg** file to:

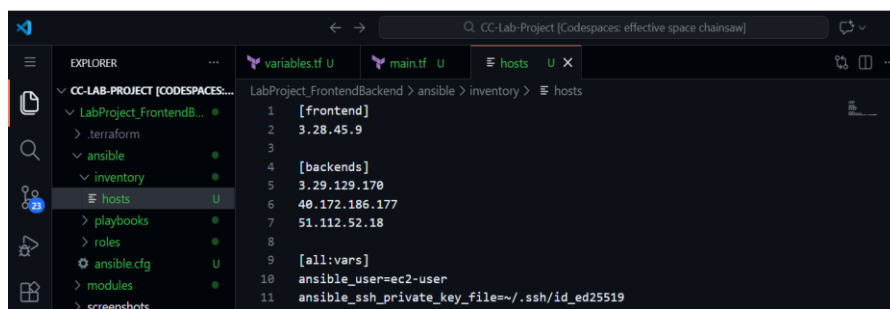
- **Disable host key checking** (so Ansible can connect to servers without asking for confirmation)
- **Set the Python interpreter** (so Ansible knows which Python version to use on the servers)

This made Ansible work **smoothly on all servers**.



```
1 [defaults]
2 host_key_checking = False
3 interpreter_python = /usr/bin/python3
```

Inventory file to define frontend and backend groups



```
1 [frontend]
2 3.28.45.9
3
4 [backends]
5 3.29.129.170
6 40.172.186.177
7 51.112.52.18
8
9 [all:vars]
10 ansible_user=ec2-user
11 ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

Inventory and Backend Role Configuration

The Ansible inventory was organized into:

- **One frontend host**
- **Three backend hosts**

Backend Role – Apache HTTPD Configuration

I created a **backend Ansible role** to set up all backend servers.

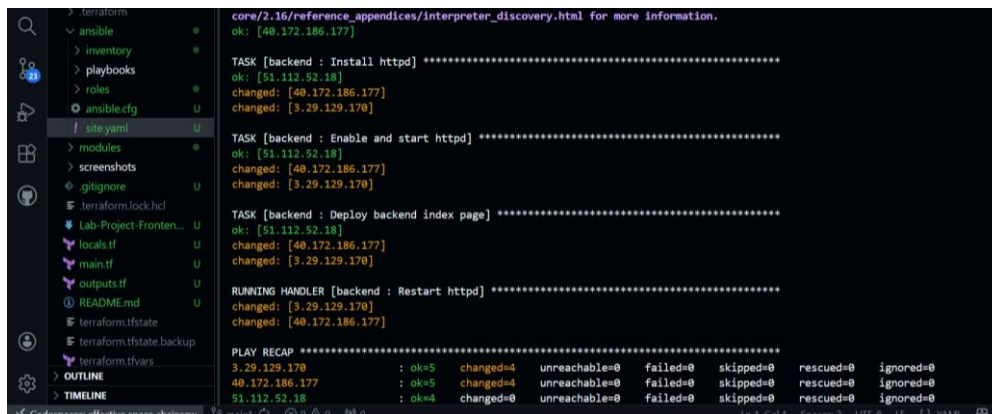
This role:

- **Installed Apache HTTPD**
- **Started and enabled the HTTPD service**
- **Deployed a different HTML page on each backend server**

I applied the configuration using this command:

`ansible-playbook -i inventory/hosts site.yaml`

This made sure all backend servers were **properly configured automatically**.



```
core/2.16/reference_appendices/interpreter_discovery.html for more information.
ok: [40.172.186.177]

TASK [backend : Install httpd] *****
ok: [51.112.52.18]
changed: [40.172.186.177]
changed: [3.29.129.170]

TASK [backend : Enable and start httpd] *****
ok: [51.112.52.18]
changed: [40.172.186.177]
changed: [3.29.129.170]

TASK [backend : Deploy backend index page] *****
ok: [51.112.52.18]
changed: [40.172.186.177]
changed: [3.29.129.170]

RUNNING HANDLER [backend : Restart httpd] *****
changed: [3.29.129.170]
changed: [40.172.186.177]

PLAY RECAP *****
3.29.129.170 : ok=5 changed=4 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
40.172.186.177 : ok=5 changed=4 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
51.112.52.18 : ok=4 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

On each backend server's web page, I displayed:

- The **hostname** of the server

- The **private IP address**

This helped to easily identify **which backend server** was responding to the request.

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Backend {{ inventory_hostname }}</title>
5 </head>
6 <body>
7   <h1>Backend server: {{ inventory_hostname }}</h1>
8   <p>Private IP: {{ ansible_default_ipv4.address | default('unknown') }}</p>
9 </body>
10 </html>

```

This made it easy to identify which backend served a request.



Frontend Role – Nginx Load Balancer Configuration

I created a **frontend Ansible role** to set up the load balancer.

This role:

- **Installed Nginx**
- **Started and enabled the Nginx service**
- **Applied the Nginx configuration** using a template file

I ran the setup using this command:

```
ansible-playbook -i inventory/hosts site.yaml
```

This automatically configured the frontend server as an **Nginx load balancer**.

```

PLAY [Configure frontend Nginx load balancer] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host 3.28.45.9 is using the discovered Python interpreter
at /usr/bin/python3.9, but future installation of another Python interpreter could
change the meaning of that path. See https://docs.ansible.com/ansible-
core/2.16/reference_appendices/interpreter_discovery.html for more information.
ok: [3.28.45.9]

TASK [frontend : Install nginx] *****
ok: [3.28.45.9]

TASK [frontend : Enable and start nginx] *****
ok: [3.28.45.9]

TASK [frontend : Deploy nginx frontend config] *****
ok: [3.28.45.9]

PLAY RECAP *****
3.28.45.9      : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
3.29.129.170  : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
40.172.186.177: ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
51.112.52.18  : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

```

Nginx Configuration

In the Nginx configuration:

- I set up an **upstream** with **two main backend servers**
- I added **one backend as a backup** in case the main ones fail
- I used **private IP addresses** of the backends instead of public IPs

This helps in **load balancing**, **failover**, and better **security**.


```
! main.yml ..handlers U nginx_frontend.confj2 U X ! site.yml U outputs.tf U ! main.yml ..tasks U
LabProject_FrontendBackend > ansible > roles > frontend > templates > nginx_frontend.confj2
1 # Nginx frontend config template
2 user nginx;
3 worker_processes auto;
4 error_log /var/log/nginx/error.log notice;
5 pid /run/nginx.pid;
6
7 events {
8     worker_connections 1024;
9 }
10
11 http {
12     log_format main '$remote_addr - $remote_user [$time_local] "$request"'
13     '$status $body_bytes_sent "$http_referer"'
14     '"$http_user_agent" "$http_x_forwarded_for"';
15
16     access_log /var/log/nginx/access.log main;
17
18     sendfile on;
19     tcp_nopush on;
20     keepalive_timeout 65;
```



Ansible Main Playbook

I created one main Ansible playbook to control the whole setup.

In this playbook:

- First, all **backend servers** were configured using the **backend role**
- After that, the **frontend server** was configured using the **frontend role**

This ensured that the backend was ready **before** the frontend was set up

```
LabProject_FrontendBackend > ansible > ! site.yml
2 # Main Ansible playbook
3 - name: Configure backend HTTPD servers
4   hosts: backends
5   become: true
6   roles:
7     - backend
8
9 - name: Configure frontend Nginx load balancer
10  hosts: frontend
11  become: true
12  vars:
13    backend1_private_ip: "{{ hostvars[groups['backends']][0]].ansible_default_ipv4.address }}"
14    backend2_private_ip: "{{ hostvars[groups['backends']][1]].ansible_default_ipv4.address }}"
15    backup_backend_private_ip: "{{ hostvars[groups['backends']][2]].ansible_default_ipv4.address }}"
16  roles:
17    - frontend
```

Terraform and Ansible Integration

To make the setup fully automatic, I connected **Ansible with Terraform**.

I used a **Terraform null resource** to run Ansible.

This setup:

- Waited until the **EC2 instances were ready**
- **Automatically ran Ansible**
- Removed the need to run Ansible **manually**

This way, the whole system was set up **by itself** without extra steps.


```

LabProject_FrontendBackend > main.tf
43 resource "null_resource" "ansible_config" {
44     triggers = {
45         frontend_ip = module.frontend.public_ips[0]
46         backend_ips  = join(",", module.backend.public_ips)
47     }
48
49     depends_on = [
50         module.frontend,
51         module.backend
52     ]
53
54     provisioner "local-exec" {
55         command = <<-EOT
56         cd ansible
57         ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook -i inventory/hosts site.yaml
58         EOT
59     }
60 }

```

Terraform apply -auto-approve

```

null_resource.ansible_config (local-exec): TASK [frontend : Deploy nginx frontend config] *****
null_resource.ansible_config (local-exec): ok: [3.28.45.9]

null_resource.ansible_config (local-exec): PLAY RECAP *****
null_resource.ansible_config (local-exec): 3.28.45.9          : ok=4   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
null_resource.ansible_config (local-exec): 3.29.129.170       : ok=4   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
null_resource.ansible_config (local-exec): 40.172.186.177     : ok=4   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
null_resource.ansible_config (local-exec): 51.112.52.18       : ok=4   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

null_resource.ansible_config: Creation complete after 18s [id=5139501220869781125]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:
backend_private_ips = [
  "10.0.1.155",
  "10.0.1.67",
  "10.0.1.253",
]
backend_public_ips = [
  "3.29.129.170",
  "40.172.186.177",
  "51.112.52.18",
]
frontend_public_ip = "3.28.45.9"

```

System Verification

1. Backend Verification



Backend server: 3.29.129.170
Private IP: 10.0.1.155

Backend server: 40.172.186.177
Private IP: 10.0.1.67

Backend server: 51.112.52.18
Private IP: 10.0.1.253

2. Load Balancing Verification



Backend server: 40.172.186.177
Private IP: 10.0.1.67

Backend server: 3.29.129.170
Private IP: 10.0.1.155

3. Failover Verification

I stopped HTTPD on both primary backends: Ssh -I ~/.ssh/id_ed25519 ec2-user@3.29.129.170

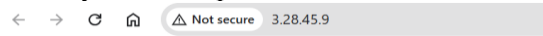
```
~/.ssh/id_ed25519
Last login: Sat Jan 24 22:03:01 2026 from 20.192.21.54
[ec2-user@ip-10-0-1-155 ~]$ sudo service httpd stop
Redirecting to /bin/systemctl stop httpd.service
[ec2-user@ip-10-0-1-155 ~]$ exit
logout
Connection to 3.29.129.170 closed.
```

Ssh -I ~/.ssh/id_ed25519 ec2-user@40.172.186.17

```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Sat Jan 24 22:03:01 2026 from 20.192.21.54
[ec2-user@ip-10-0-1-67 ~]$ sudo service httpd stop
Redirecting to /bin/systemctl stop httpd.service
[ec2-user@ip-10-0-1-67 ~]$ exit
logout
Connection to 40.172.186.177 closed.
```

Traffic was automatically served by the backup backend



Backend server: 51.112.52.18

Private IP: 10.0.1.253

Idempotence Test

I ran the **Terraform apply** command again to check idempotence.

This means making sure that running the same command again does **not change anything** if everything is already correct.

The command ran successfully:

- There were **no errors**
- Terraform made **no extra or unnecessary changes**
- The infrastructure stayed **exactly the same**

Command used:

terraform apply -auto-approve

```
module.network.aws_route_table_association.public_assoc: Refreshing state... [id=rtbassoc-0aaff9a5a405e3ecc]
module.frontend.aws_instance.this[0]: Refreshing state... [id=i-0cb49d3d9797eff94]
module.backend.aws_instance.this[1]: Refreshing state... [id=i-05e4f5bb213d108fd]
module.backend.aws_instance.this[0]: Refreshing state... [id=i-084b29708ae3dea8c]
module.backend.aws_instance.this[2]: Refreshing state... [id=i-02072643fa3f9647f]
null_resource.ansible_config: Refreshing state... [id=5139501220869781125]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no
differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

backend_private_ips = [
  "10.0.1.155",
  "10.0.1.67",
  "10.0.1.253",
]
backend_public_ips = [
  "3.29.129.170",
  "40.172.186.177",
  "51.112.52.18",
]
frontend_public_ip = "3.28.45.9"
```

Cleanup

At the end, I deleted all the AWS resources created by Terraform to avoid extra charges.

I used the destroy command to remove everything.

I checked and confirmed that:

- All EC2 instances were successfully deleted
- No resources were left running

Command used:

terraform destroy -auto-approve

```
module.backend.aws_instance.this[2]: Still destroying... [id=i-02072643fa3f9647f, 01m00s elapsed]
module.backend.aws_instance.this[0]: Destruction complete after 1m0s
module.backend.aws_instance.this[2]: Destruction complete after 1m0s
module.network.aws_subnet.public: Destroying... [id=subnet-0773b10818be1e65b]
module.network.aws_security_group.main: Destroying... [id=sg-0f2c6db89e599ee73]
module.network.aws_subnet.public: Destruction complete after 1s
module.network.aws_security_group.main: Destruction complete after 1s
module.network.aws_vpc.main: Destroying... [id=vpc-0cf0f87d7cc468495]
module.network.aws_vpc.main: Destruction complete after 0s

Destroy complete! Resources: 12 destroyed.
```

No infrastructure remained

Security Groups

Security Groups (1) Info

Find security groups by attribute or tag

Name	Security group ID	Security group name	VPC ID	Description
-	sg-0567ae2c936d1bb1c	default	vpc-05883252ecb09ac3e	default VPC security group

Conclusion

In this project, I created a system where the **frontend and backend work automatically without manual setup.**

I used **Terraform** to create the servers and **Ansible roles** to configure them.

This project shows that:

- The infrastructure is **well-planned and organized**
- Different tasks are handled using **separate roles**
- The system keeps working even if **one server fails**
- Everything is set up using a **real-world automation process**